LeetCode 2064: Minimized Maximum of Products Distributed to Any Store

1. Problem Title & Link

Title: LeetCode 2064: Minimized Maximum of Products Distributed to Any Store

Link: https://leetcode.com/problems/minimized-maximum-of-products-distributed-to-any-store/

2. Problem Statement (Short Summary)

You are given:

- n stores
- quantities[i] = number of products of type i

Rules:

- Each store can receive **only one product type**
- A product type can be distributed across multiple stores

Goal:

Minimize the maximum number of products assigned to any store.

Return the minimum possible value.

3. Examples (Input → Output)

Example 1

Input:

n = 6

quantities = [11,6]

Output:

3

Explanation:

Distribute:

11 → 3,3,3,2

6 → 3,3

Maximum products in a store:

3

Example 2

Input:

n = 7

quantities = [15,10,10]

Output:

5

Example 3

Input:

n = 1

quantities = [100000]

Output:

100000

4. Constraints

- $1 \le n \le 10^5$
- $1 \le quantities.length \le 10^5$
- $1 \le quantities[i] \le 10^5$
- $quantities.length \le n$

Need better than simulation.

5. Core Concept (Pattern / Topic)

Binary Search on Answer

Related Patterns:

- Minimize Maximum
- Capacity Problems
- Feasibility Checking

6. Thought Process (Step-by-Step Explanation)

Brute Force

Try every possible maximum value:

1

2

3

4

...

max(quantities)

For each value, simulate distribution.

Too slow.

Key Observation

Question asks:

Minimize the Maximum

This is a strong hint for:

Binary Search on Answer

Suppose We Guess

maxProducts = 5

Question becomes:

Can all products be distributed

such that no store gets

more than 5 products?

Stores Needed

For a quantity:

quantity = 11

Maximum per store:

5

Stores needed:

ceil(11/5)

=

3

Distribution:

5 + 5 + 1

If total stores needed:

<= n

Then answer is possible.

Otherwise impossible.

Why Binary Search Works

If maximum value:

5

works,

then

6,7,8,9...

also work.

Monotonic property exists.

Therefore:

Binary Search on Answer

7. Visual / Intuition Diagram (ASCII Diagram)

```
Example:
n = 6
quantities = [11,6]
Check:
mid = 3
For 11:
3 3 3 2

Stores = 4
For 6:
3 3

Stores = 2
Total:
4 + 2 = 6 stores
Valid.
```

8. Pseudocode (Language Independent)

```
low = 1

high = max(quantities)

while low <= high:

    mid = (low + high)//2

    if feasible(mid):

        answer = mid
        high = mid - 1

    else:

        low = mid + 1

return answer
Feasible:
storesNeeded = 0

for quantity:

    storesNeeded += ceil(quantity / mid)

return storesNeeded <= n
```

9. Code Implementation

✅ **Python**

```python
import math

class Solution:

    def minimizedMaximum(self, n, quantities):

        def feasible(limit):

            stores = 0

            for q in quantities:
                stores += math.ceil(q / limit)

            return stores <= n

        low = 1
        high = max(quantities)

        answer = high

        while low <= high:

            mid = (low + high) // 2

            if feasible(mid):
                answer = mid
                high = mid - 1
            else:
                low = mid + 1

        return answer
```

✅ **Java**

```java
class Solution {

    public int minimizedMaximum(
        int n,
        int[] quantities
    ) {

        int low = 1;

        int high = 0;

        for (int q : quantities)
```

```
            high = Math.max(high, q);

        int answer = high;

        while (low <= high) {

            int mid = low + (high - low) / 2;

            if (feasible(quantities, n, mid)) {

                answer = mid;
                high = mid - 1;

            } else {

                low = mid + 1;
            }
        }

        return answer;
    }

    private boolean feasible(
        int[] quantities,
        int n,
        int limit
    ) {

        long storesNeeded = 0;

        for (int q : quantities) {

            storesNeeded +=
                (q + limit - 1) / limit;

            if (storesNeeded > n)
                return false;
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity

Binary Search:

O(log(maxQuantity))

Feasibility Check:

O(m)

where:

m = quantities.length

Overall:

O(m log(maxQuantity))

Space Complexity

O(1)

11. Common Mistakes / Edge Cases

Mistake 1

Using floor division:

q // limit

Wrong.

Need:

ceil(q / limit)

Mistake 2

Binary searching stores instead of answer.

Mistake 3

Not handling:

q % limit != 0

properly.

12. Detailed Dry Run (Step-by-Step Table)

Input:

n = 6

quantities = [11,6]

Check:

mid = 3

Quantity	Stores Needed
11	4
6	2

Total:

6

Since:

6 <= n

Valid.

Check:

mid = 2

Quantity	Stores Needed
11	6
6	3

Total:

9

Since:

9 > n

Invalid.

Answer:

3

13. Common Use Cases (Real-Life / Interview)

Warehouse Distribution

Distribute products among warehouses while minimizing peak load.

Cloud Servers

Assign requests to servers while minimizing maximum server load.

Logistics

Split shipments across delivery hubs.

Manufacturing

Distribute work among machines evenly.

14. Common Traps (Important!)

- Using floor instead of ceil
- Wrong binary search boundaries
- Overflow in store counting

- Binary searching quantity indices

15. Builds To (Related LeetCode Problems)

1. LC 875 — Koko Eating Bananas
2. LC 1011 — Capacity To Ship Packages
3. LC 1283 — Smallest Divisor Given Threshold
4. LC 1231 — Divide Chocolate
5. LC 2064 — Minimized Maximum of Products Distributed to Any Store

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	O(maxQ × m)	O(1)	Slow
Greedy Simulation	Very Large	O(1)	Inefficient
Binary Search + Feasibility	O(m log(maxQ))	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

If a maximum store load x is feasible, then any value greater than x is also feasible. This monotonic property enables binary search. For each candidate value, we calculate how many stores are required using ceiling division.

18. Variations / Follow-Up Questions

Follow-Up 1

What if stores can contain multiple product types?

The solution changes completely and becomes a load-balancing problem.

Follow-Up 2

What if products arrive dynamically?

Use heaps and online allocation strategies.

Follow-Up 3

What if we want to maximize the minimum load instead?

Binary search still applies, but feasibility logic changes.